```
            )
        if len(pts) < 4:
            return None

        t0 = pts[0][0]
        xs = [(p[0] - t0).total_seconds() for p in pts]
        lat_vals = [p[1] for p in pts]
        lon_vals = [p[2] for p in pts]
        alt_vals = [p[3] for p in pts]

        lat_fit = self._fit_linear_regression_1d(xs, lat_vals)
        lon_fit = self._fit_linear_regression_1d(xs, lon_vals)
        alt_fit = self._fit_linear_regression_1d(xs, alt_vals)
        if lat_fit is None or lon_fit is None or alt_fit is None:
            return None

        t_last = xs[-1]
        t_future = t_last + float(age_sec)
        lat_pred = lat_fit[0] + lat_fit[1] * t_future
        lon_pred = lon_fit[0] + lon_fit[1] * t_future
        alt_pred_m = max(0.0, alt_fit[0] + alt_fit[1] * t_future)

        # Derive heading/speed from regression slopes (deg/s -> meters/s approx).
        lat_rate = lat_fit[1]
        lon_rate = lon_fit[1]
        lat_now = lat_fit[0] + lat_fit[1] * t_last
        lon_now = lon_fit[0] + lon_fit[1] * t_last
        lat_next = lat_now + lat_rate
        lon_next = lon_now + lon_rate
        speed_cms = int(round(_haversine_m(lat_now, lon_now, lat_next, lon_next) * 100.0))
        heading_de2 = int(round(_bearing_deg(lat_now, lon_now, lat_next, lon_next) * 100.0))
        vs_cms = int(round(alt_fit[1] * 100.0))

        # Keep fallback stability if regression outputs become non-finite.
        if not (math.isfinite(lat_pred) and math.isfinite(lon_pred)):
            return None
        if not math.isfinite(alt_pred_m):
            alt_pred_m = float(fallback_alt_m)
        if speed_cms < 0:
            speed_cms = 0
        if heading_de2 < 0:
            heading_de2 = 0

        return {
            "lat": lat_pred if math.isfinite(lat_pred) else fallback_lat,
            "lon": lon_pred if math.isfinite(lon_pred) else fallback_lon,
            "alt_m": alt_pred_m,
            "speed_cms": speed_cms,
            "heading_de2": heading_de2,
            "vs_cms": vs_cms,
        }

    def build_predicted_rows(self, active_icao_set=None, max_prediction_age_sec=120, limit=1000):
        """
        Build short-horizon predicted rows for flights that dropped from live feed.
        Predictions are output-only (not written into training tables).
        """
        if not self.enabled:
            return []

        active_icao_set = {str(x).strip().upper() for x in (active_icao_set or set()) if str(x).strip()}
        max_prediction_age_sec = max(1, int(max_prediction_age_sec))
        limit = max(1, int(limit))

        now_dt = datetime.now(timezone.utc)

        cur = self._conn.cursor()
        cur.execute(
            """
            SELECT
                flight_key,
                icao_address,
                callsign,
```